

Sessão de apoio ao projeto



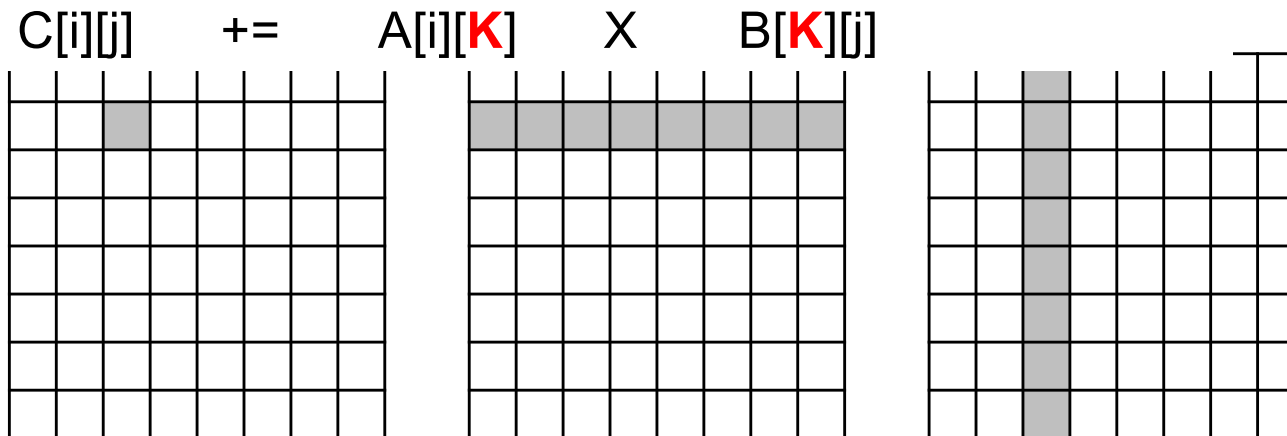
- **Estão previstas duas sessões para hoje:**
16h-18h e 18h-19h, na sala Ed1. 2.21
- **Nestas sessões:**
 - Será apresentada a infraestrutura computacional onde terão que ser executados os trabalhos da disciplina
(Deucalion, nós A64FX – 48 core – ARM)
 - conjunto de boas práticas
 - Objetivo: aceder à plataforma e analisar a escalabilidade de um “benchmark” simples de multiplicação de matrizes
- Numa próxima sessão está prevista a apresentação e apoio ao arranque do projeto.

[Parallel] Matrix Multiplication

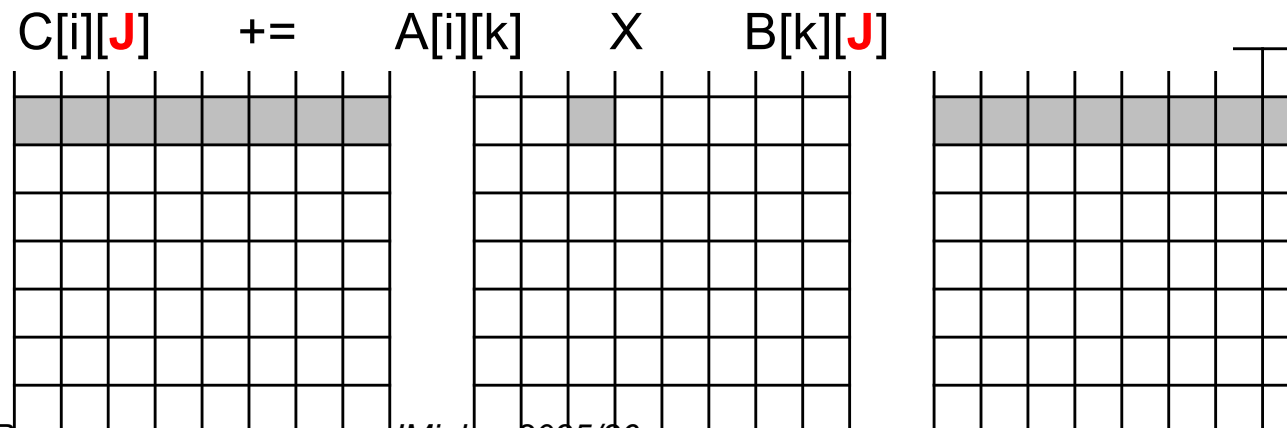


- Best PL question: **data locality** and loop ordering
 - 6 possible alternatives (3x2x1 possible loop orders) => 3 options

1. DOT **xxK** ordering



2. AXPYv1 **xxJ** ordering

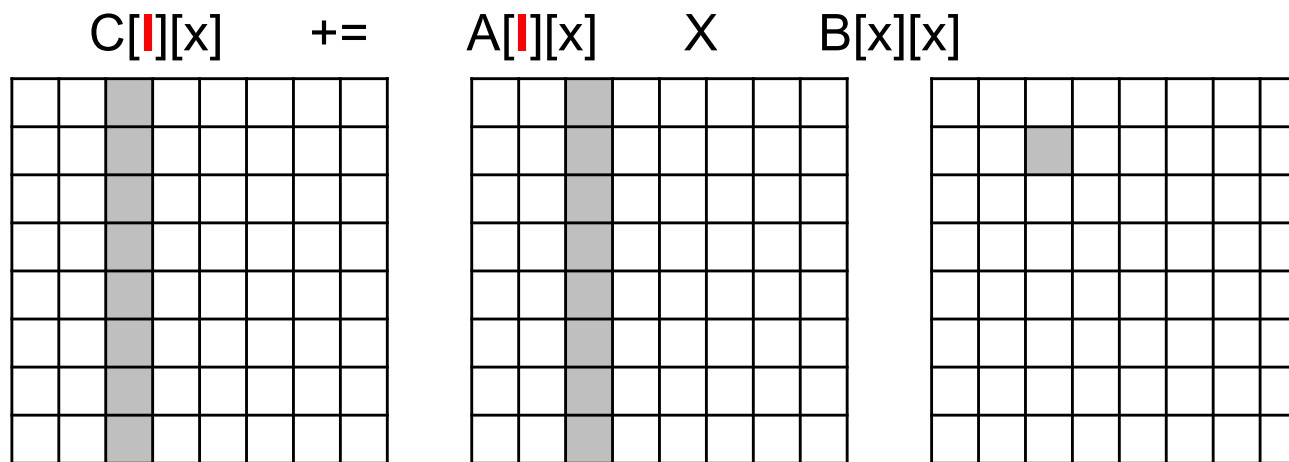


[Parallel] Matrix Multiplication

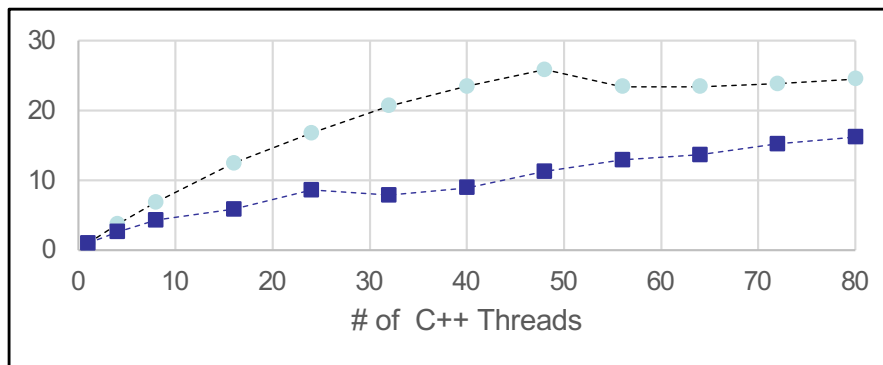


- Best PL question: **data locality** and loop ordering (cont)

3. AXPYv3 **xxl** ordering



- Scalability analysis of the DOT version: **ijk** vs **jik** ordering



$$C[i=tid][j] += A[i][k] \times B[k][j]$$

$$C[i][j=tid] += A[i][k] \times B[k][j]$$

Note: A64FX machine



Processing Accelerators

MEI / MCA /MMC

2025/26

J.L. Sobral

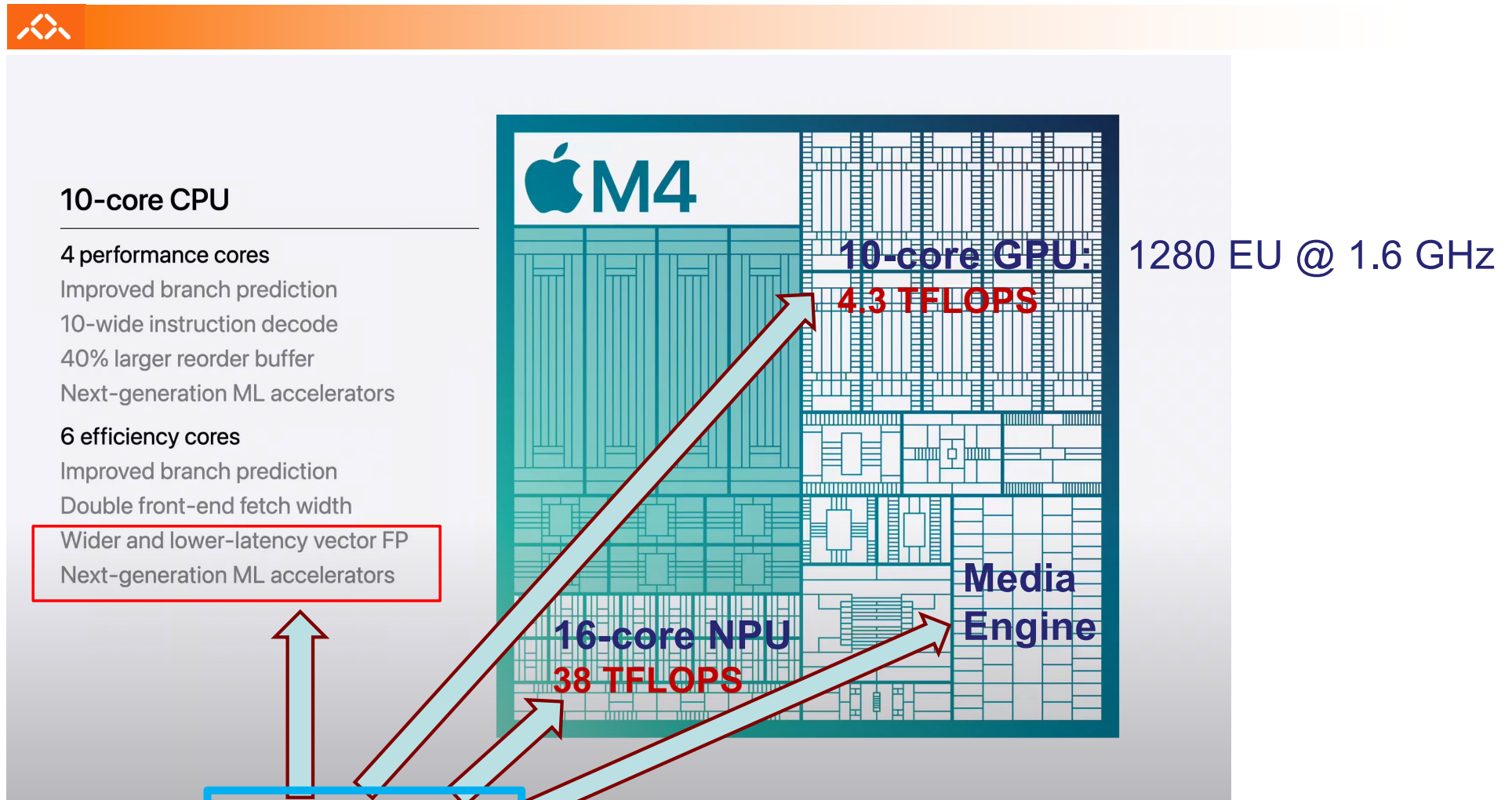
What is a Processing accelerator?



“**Processing accelerators** are specialized hardware components that perform specific computational tasks much faster and more efficiently than a general-purpose CPU. Common examples include [Graphics Processing Units \(GPUs\)](#) for parallel tasks like AI and image processing, [Tensor Processing Units \(TPUs\)](#) for AI workloads, and [Field-Programmable Gate Arrays \(FPGAs\)](#) which offer flexibility. These accelerators are crucial for modern applications like AI, machine learning, and high-performance computing that require more speed and power than traditional processors can provide alone.”

Vista geral IA (google)

SoC Apple M4: Laptop or HPC system?



Parallel + specialized HW: We will also discuss how modern systems can utilize specialized processing units to achieve high levels of efficiency

 EPOCH AI

Performance (OP/s)

- TF32 (TensorFloat-32)

- TF16 (Tensor-FP16/BF16)

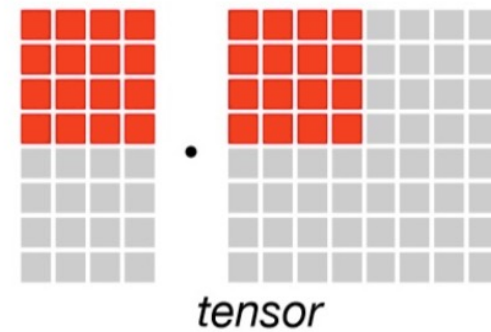
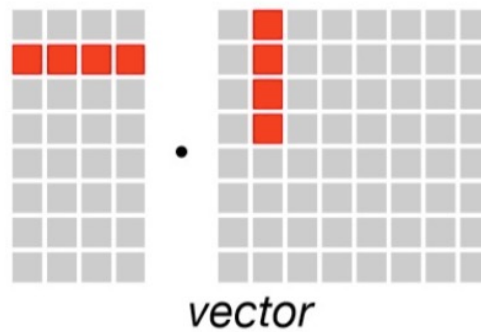
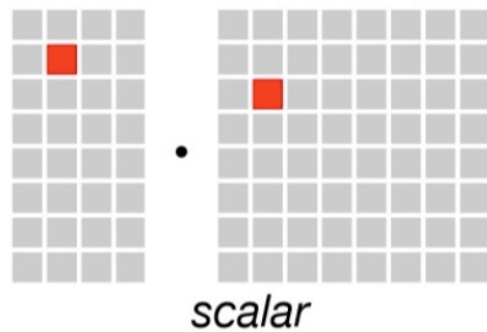
- INT8



Compute primitive (smallest unit) in **CPU, GPU and TPU/NPU**

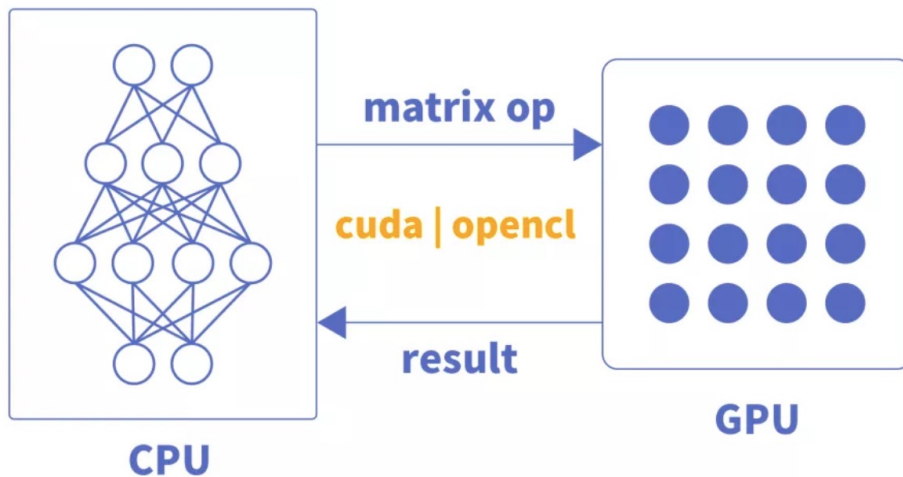


Compute Primitive

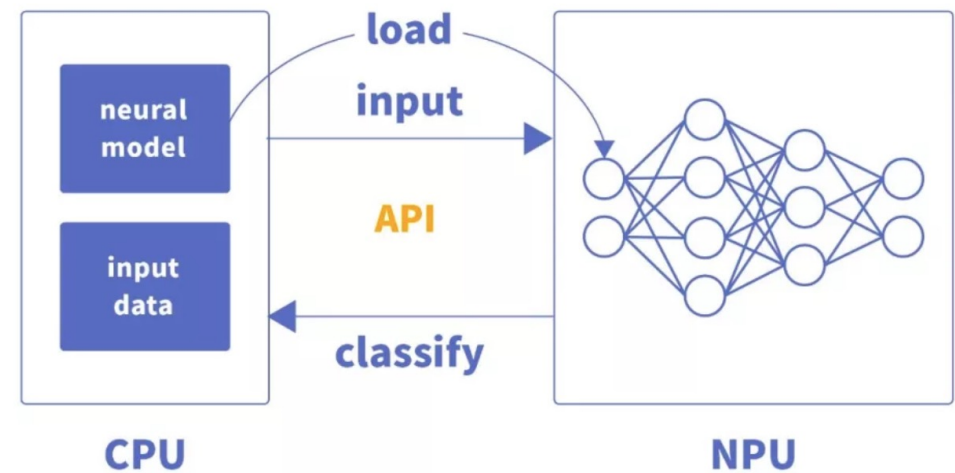


Vector architectures and SIMD parallelism

neural network



neural network



Vector Architectures (& SIMD parallelism)

- Basic idea:
 - Read sets of data elements (*gather from memory*) into “vector registers”
 - Operate on those registers
 - Disperse the results back into memory (*scatter*)
- Registers are controlled by the compiler
 - Used to hide memory latency
 - Leverage memory bandwidth

SIMD Parallelism

- Key principle underlying:
 - Vector architectures
 - SIMD extensions in modern CPUs
 - Graphics Processor Units (GPUs) *(in another set of slides)*

Flynn's Taxonomy of Computers *

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD: Intel Pentium 4	SIMD: SSE instructions of x86
	Multiple	MISD: No examples today	MIMD: Multicore devices

* Mike Flynn, "Very High-Speed Computing Systems", Proc. of IEEE, 1966



Single Instruction Multiple Data - SIMD*

A total of 2 floating-point instructions

```
for (i=0; i<3; i++)  
{  
    a[i] = 2 * b[i];  
    c[i] = 3 + d[i];  
}
```

Vectorization

a[0] = 2 * b[0]
a[1] = 2 * b[1]
a[2] = 2 * b[2]
a[3] = 2 * b[3]

vfmul 2,b,a

c[0] = 3 + d[0]
c[1] = 3 + d[1]
c[2] = 3 + d[2]
c[3] = 3 + d[3]

vfadd 3,d,c

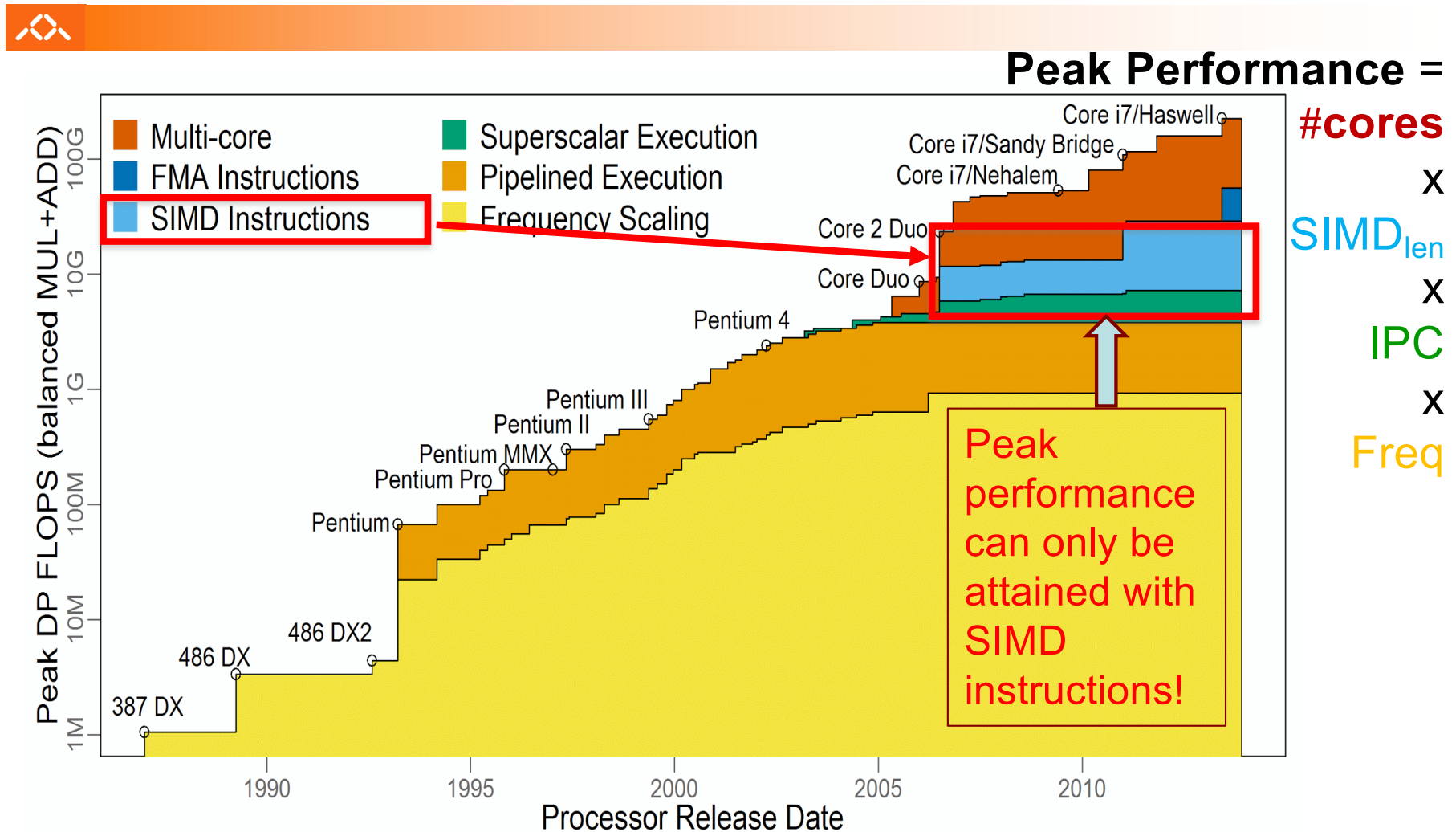
*) This depends on what the core supports, if anything.



Introduction to SIMD

- SIMD architectures can exploit significant **data-level parallelism** for:
 - Matrix-oriented scientific computing
 - Media-oriented image and sound processors
 - Machine learning algorithms
- SIMD is more energy efficient than MIMD
 - Only needs to fetch one instruction per data operation
 - Makes SIMD attractive for personal mobile devices
- SIMD allows programmer to continue to think sequentially

Evolution of Peak performance

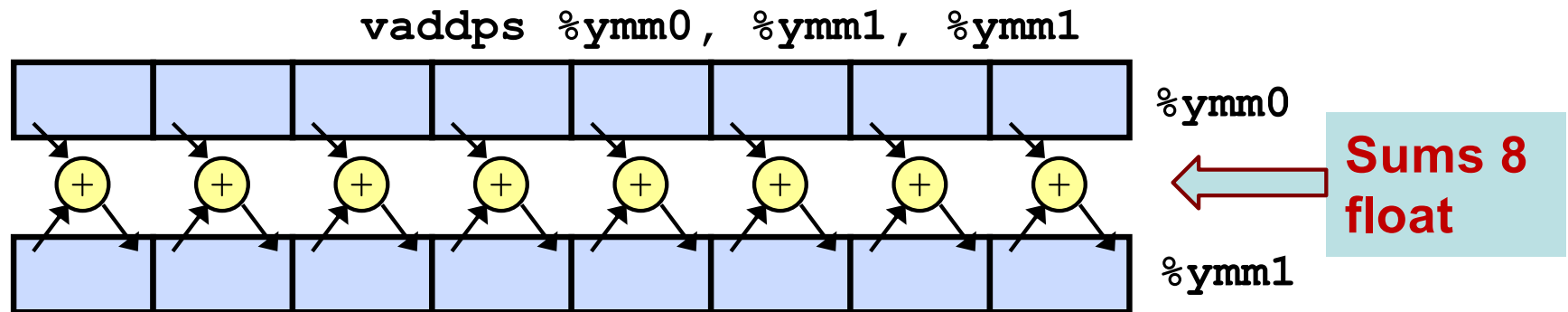


$$T_{\text{exe}}(\text{ideal}) = \#I_{\text{vect}} \times \text{CPI} \times T_{\text{cc}} / \# \text{cores} \quad (\#I_{\text{vect}} = \#I / \text{SIMD}_{\text{len}})$$

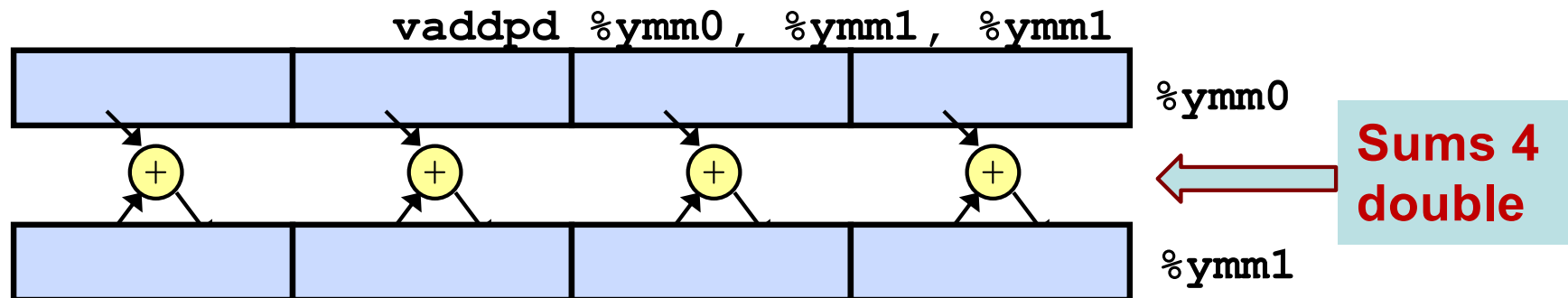
SIMD Operations on Intel x64_x86



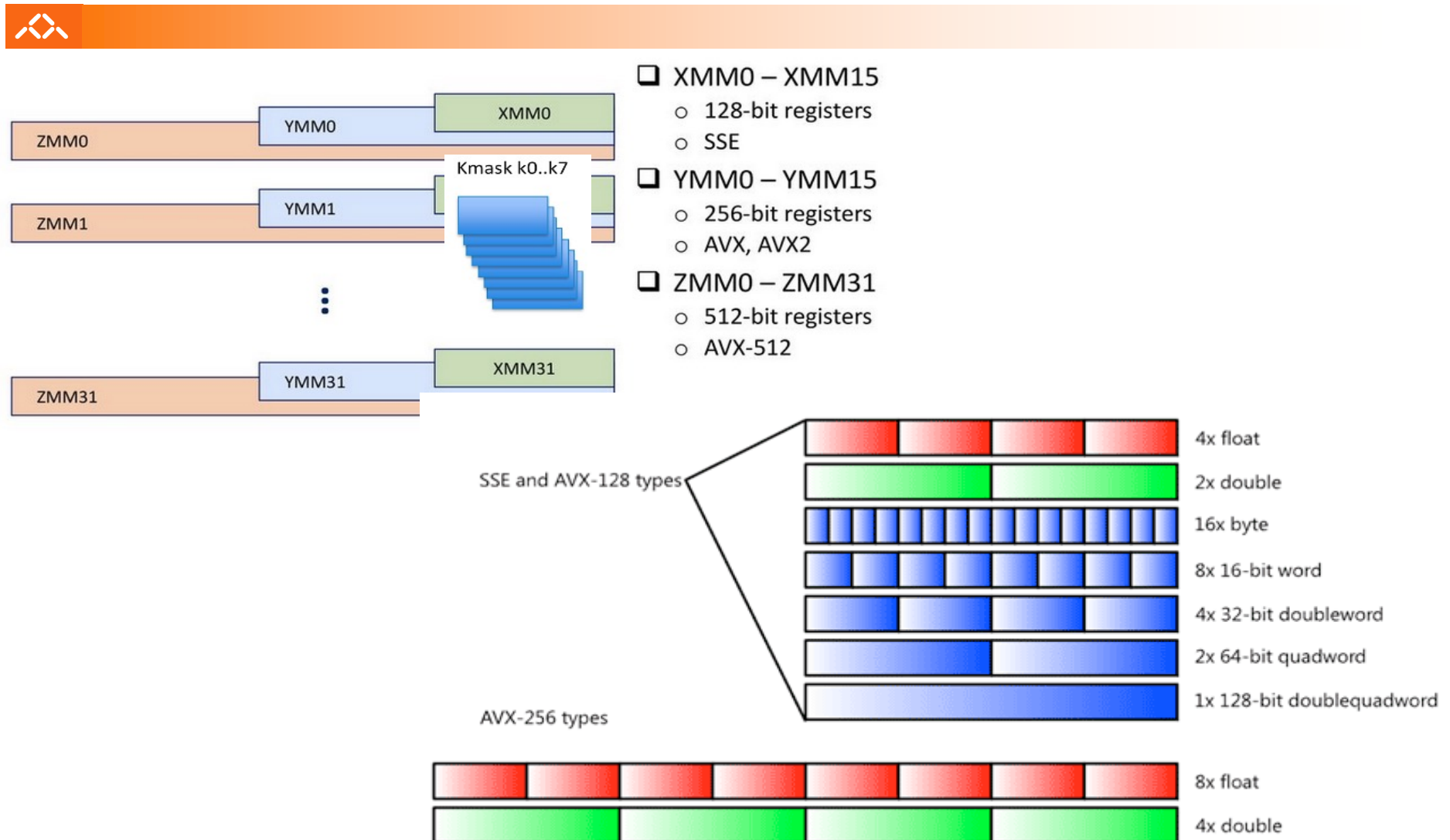
- SIMD operation over **float** (each register contains 8 floats)



- SIMD operation over **double** (each register contains 4 double)



Registers for vector processing in Intel x64



Vector Programming (& challenges)

- Compilers are a key element to give hints on whether a code section will vectorize or not

- Check if loop iterations have **data dependencies** and/or if...then statements, otherwise vectorization is compromised

- Vector architectures have a too high cost, but simpler variants are currently available on off-the-shelf devices, as SIMD extensions to the scalar PUs; **however**:

- most do not support **non-unit stride**
=> care must be taken in the design of data structures (data layout)
- same applies for **mask register**,
- **gather-scatter...**



No data dependencies

- Vector prog. applies only to data-parallelism (e.g. map pattern)



Deal with control dependencies

- common solutions:
 - predication => additional work
 - mask registers => unused PEs

Support for arbitrary vector lengths

- when the vector registers size does not match the vector length



Maximize data locality

- maximize spatial locality (next slides)
- data alignment in memory

NOTE: SIMD consumes much more data per instruction!

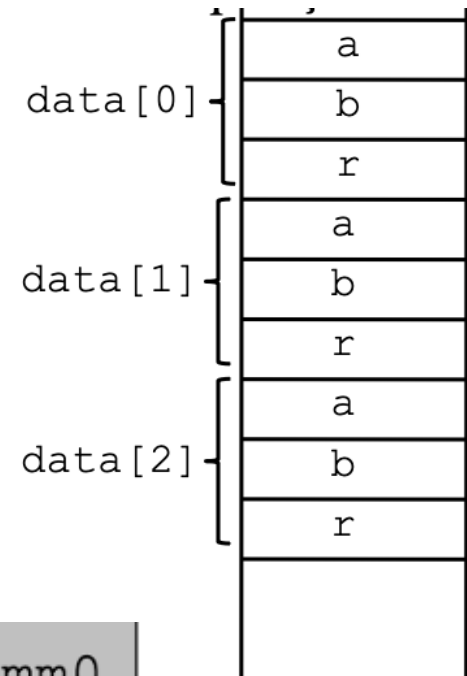
Data layout: AoS vs. SoA (1)



A typical C code program defines the following data layout as an Array of Structures (AoS):

```
struct {  
    float a, b, r;  
} data[100];
```

This array will be
stored in memory
like this:



Now consider this instruction:

```
movaps (%eax, %ecx, 4), %xmm0
```

It expects to find a vector of 4 floats in consecutive memory cells: namely `a[0]`, `a[1]`, `a[2]`, `a[3]` but it finds a non-unit stride...

Conclusion: the compiler will not vectorize this code!

Data layout: AoS vs. SoA (2)

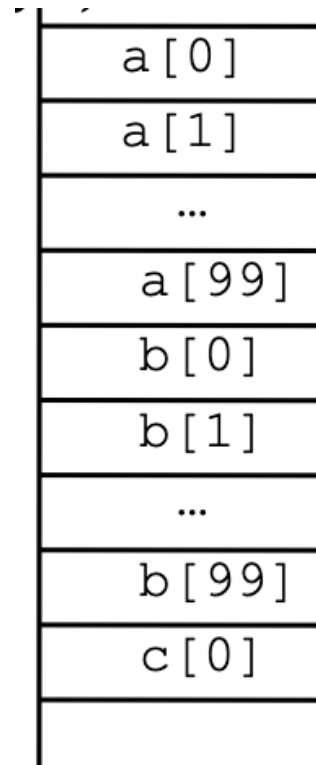


Solution

The C code should redefine the data layout as a Structure of Arrays (SoA):

```
struct {  
    float a[100], b[100], r[100];  
} data;
```

Now the structure of arrays is
stored in memory like this:



Conclusion: the compiler now vectorizes that code!

Intel Advanced Matrix Extension (AMX)

